

MODULE 25

Service and Repository Layers

Design, build, and test the Service and Repository layers that keep enterprise Spring applications clean, testable, and ready to scale.





WHY LAYERED ARCHITECTURE MATTERS

Layered architecture organizes an application into clear, logical layers with well-defined responsibilities and boundaries.

It helps teams build robust, maintainable, and scalable applications that can evolve with business needs.

THE LAYERED ARCHITECTURE MODEL

Layer	Role
Presentation	Handles user requests and responses (Controllers, APIs).
Service	Contains business logic and orchestrates operations.
Repository	Handles data access and interacts with the database.
Database	Persists and manages application data.

KEY BENEFITS

- **Separation of Concerns** — each layer has a single responsibility, reducing complexity.
- **Maintainability** — changes in one layer have minimal impact on others.
- **Scalability** — layers can be scaled independently based on demand.
- **Testability** — layers can be unit tested in isolation for better quality.
- **Team Productivity** — teams can work on different layers in parallel with clear contracts.

WITHOUT LAYERED ARCHITECTURE

Tight Coupling

Difficult Maintenance

Hard to Test

Poor Scalability

Slow Delivery

KEY TAKEAWAY Layered architecture brings structure, clarity, and consistency -- build it right, build it once, scale with confidence.

MODULE LEARNING OBJECTIVES

By the end of this module, you will be able to understand, design, and implement Service and Repository layers following enterprise best practices.

- **Explain Layered Architecture** — its importance and its role in building enterprise applications.
- **Describe the Pattern** — the Controller-Service-Repository pattern and the flow of requests through each layer.
- **Define the Service Layer** — explain how it manages business logic and application rules.
- **Implement Service Components** — with a focus on reusability, testability, and maintainability.
- **Manage Service Dependencies** — explain dependencies in the service layer and how to manage them effectively.
- **Design Repository Components** — that provide clean, efficient, and secure data access.
- **Apply Repository Best Practices** — including exception handling and performance.
- **Understand Layer Communication** — how layers communicate and depend on each other.
- **Apply Enterprise Boundaries** — identify and apply design boundaries for transactions, validation, and error handling.
- **Build a Layered Application** — using Service and Repository layers following best practices.

KEY TAKEAWAY Mastering Service and Repository layers is essential for building scalable, maintainable, and enterprise-ready applications.

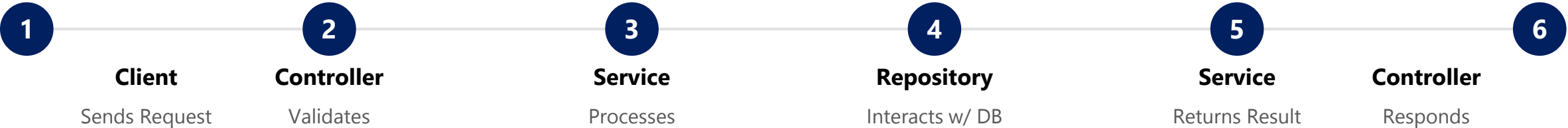
CONTROLLER-SERVICE-REPOSITORY PATTERN (1 OF 2)

The pattern organizes an application into three distinct layers with well-defined responsibilities and clear interactions.

"Each layer has a single focus and communicates only with the layer next to it. This separation leads to clean code, high maintainability, and easier testing."

Layer	Responsibility
Controller Layer	Handles HTTP requests and responses. Validates input and delegates to the Service.
Service Layer	Contains business logic and application rules. Orchestrates operations and calls the Repository.
Repository Layer	Handles data access and persistence. Abstracts and encapsulates database operations.
Database	Stores and manages application data.

FLOW OF OPERATION



KEY TAKEAWAY Each layer talks only to its neighbor -- Controller to Service, Service to Repository -- never skipping a layer.



CONTROLLER-SERVICE-REPOSITORY PATTERN (2 OF 2)

Layer responsibilities in detail, and why enterprise teams standardize on this pattern.

CONTROLLER

- Expose REST endpoints.
- Validate request data.
- Handle authentication/authorization.
- Return appropriate responses.

SERVICE

- Implement business logic.
- Apply business rules.
- Manage transactions (if needed).
- Call repository for data operations.

REPOSITORY

- Perform CRUD operations.
- Use ORM / SQL / query APIs.
- Hide data access complexity.
- Return domain data to Service.

WHY THIS PATTERN?



Separation of Concerns

Each layer changes for one reason only.



Easier Maintenance

Refactoring one layer rarely breaks another.



Improved Testability

Each layer can be tested in isolation.



Scalability & Flexibility

Layers scale and evolve independently.



Better Collaboration

Teams work across layers with clear contracts.

KEY TAKEAWAY A well-structured application today leads to an enterprise-ready system tomorrow.

SEPARATION OF CONCERNS

Separation of Concerns (SoC) is a design principle that divides an application into layers, where each layer has a specific responsibility.

"Each layer should have one reason to change." -- Software Engineering Best Practice

Layer	Focuses On	Does NOT Do
Controller	Handling web requests and responses.	Does not contain business logic or data access code.
Service	Business logic and orchestration.	Does not handle HTTP or direct database access.
Repository	Data access and persistence.	Does not contain business rules or request handling.

BENEFITS OF SEPARATION OF CONCERNS



Maintainability

Changes in one layer have minimal impact on others.



Testability

Each layer can be tested independently in isolation.



Scalability

Layers can be scaled independently based on demand.



Flexibility

Easier to adapt to new requirements and technologies.



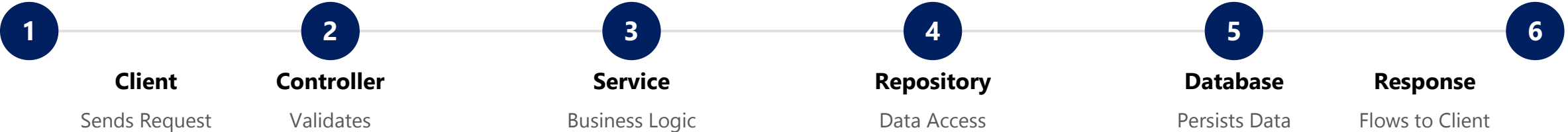
Team Productivity

Teams work on different layers in parallel.

KEY TAKEAWAY Separation of Concerns leads to clean code, lower complexity, and enterprise-ready applications.

REQUEST PROCESSING FLOW

A well-defined flow ensures every request moves through the application layers in a controlled and predictable manner.



PER-LAYER RESPONSIBILITIES

Layer	What Happens Here
Controller	Receives HTTP request, validates input, authenticates/authorizes, maps to service call, returns response.
Service	Executes business rules, performs validations, orchestrates operations, manages transactions, calls Repository.
Repository	Abstracts data source complexity, executes CRUD operations, returns domain data to Service.
Database	Stores and manages application data.

KEY TAKEAWAY From client request to final response, each layer plays a specific role in delivering reliable business outcomes.

WHAT IS A SERVICE LAYER? (1 OF 2)

The Service Layer is the heart of the application -- it contains the business logic and orchestrates operations.

It acts as an intermediary between the Controller and the Repository layer, ensuring that business requirements are met before any data is accessed or modified.

KEY CHARACTERISTICS

- **Business Logic** — encapsulates core business rules and computations.
- **Validation** — validates inputs and enforces business constraints.
- **Orchestration** — coordinates multiple operations and components.
- **Reusability** — provides reusable operations across multiple controllers.
- **Transaction Management** — manages transactions to ensure data consistency.
- **Abstraction** — hides data access details from upper layers.

KEY TAKEAWAY The Service Layer holds the business rules and orchestrates work. It is independent of how data is stored and focuses only on what the business requires.

WHAT IS A SERVICE LAYER? (2 OF 2)

Where the Service Layer fits in the request flow, and a worked example: placing an order.



SCENARIO: PLACE ORDER

- Validate products and stock.
- Calculate order total and discounts.
- Apply business rules (e.g. credit limit).
- Create order and update inventory.
- Call Repository Layer to save data.
- Return result to Controller.

OrderService.java

```
@Service
public class OrderService {
    public OrderResponse placeOrder(OrderRequest request) {
        // Business logic
        // Validation
        // Orchestration
        // Call repository
        return response;
    }
}
```

KEY TAKEAWAY A well-designed service layer makes your application clean, maintainable, and scalable.

BUSINESS LOGIC MANAGEMENT (1 OF 2)

The Service Layer is responsible for managing the business logic of the application.

It ensures that all business rules are applied consistently, transactions are coordinated, and data is used correctly to produce reliable results.

WHAT BELONGS IN BUSINESS LOGIC?

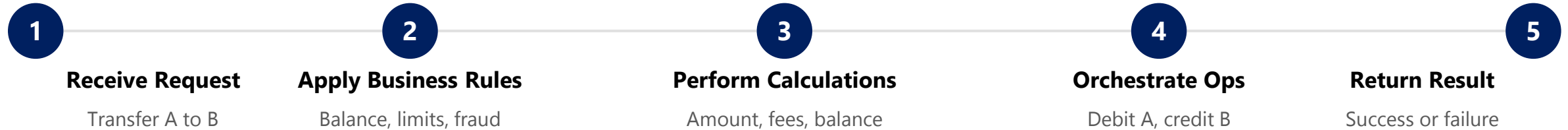
Category	Description
Business Rules	Implement domain rules (e.g. credit limit check).
Validations	Validate input and business constraints.
Calculations	Perform pricing, tax, discount, interest, and other calculations.
Workflow Orchestration	Coordinate multiple operations and services.
Transaction Management	Ensure atomicity and data consistency.

KEY TAKEAWAY Good business logic management leads to correct, consistent, and maintainable enterprise applications.

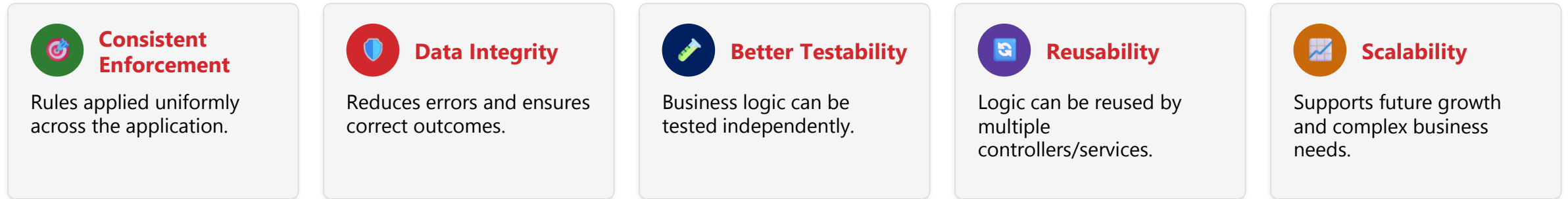
BUSINESS LOGIC MANAGEMENT (2 OF 2)

Worked example: a fund transfer service, and the benefits of managing business logic well.

EXAMPLE: FUND TRANSFER SERVICE



BENEFITS OF EFFECTIVE BUSINESS LOGIC MANAGEMENT



KEY TAKEAWAY Keep business logic in the Service Layer -- not in Controllers or Repositories -- for correct, consistent, and maintainable enterprise applications.

SERVICE DESIGN BEST PRACTICES (1 OF 2)

Well-designed services are maintainable, testable, scalable, and aligned with business needs.

#	Practice	What It Means
1	Single Responsibility	Each service should have one well-defined business responsibility.
2	Business Focused	Design services around business capabilities, not technical operations.
3	Loosely Coupled	Minimize dependencies between services; use interfaces.
4	High Cohesion	Keep related operations together in the same service.
5	Transaction Management	Use transactions appropriately; keep them short and within service boundaries.
6	Validate at the Service Layer	Enforce business rules and validate inputs before accessing data.
7	Exception Handling	Handle exceptions gracefully and throw meaningful business exceptions.
8	Stateless & Reusable	Design stateless services whenever possible to improve scalability.

KEY TAKEAWAY A good service is cohesive, focused on business value, independent of how data is stored, and easy to evolve.

SERVICE DESIGN BEST PRACTICES (2 OF 2)

A design checklist, plus a good and a bad service example, side by side.

SERVICE DESIGN CHECKLIST

- Does the service have a single, clear business purpose?
- Are service methods named based on business actions?
- Are input parameters validated?
- Does the service enforce business rules?
- Are repository dependencies injected (not created inside the service)?
- Are transactions defined at the right boundaries?
- Are exceptions logged and rethrown meaningfully?
- Is the service easy to unit test (mockable dependencies)?

Good: OrderService

```
placeOrder(request)
validateStock(request)
calculateTotal(request)
applyDiscount(request)
saveOrder(order)
```

Anti-Pattern: BadOrderService (avoid)

```
placeOrder()
sendEmail()
generateReport()
callPaymentGateway()
logToDatabaseDirectly()
```

KEY TAKEAWAY Follow these best practices to build clean, maintainable, scalable, and business-aligned services.

SERVICE DEPENDENCIES (1 OF 2)

A Service Layer does not work in isolation -- it depends on other components to perform its responsibilities.

Managing these dependencies properly leads to loosely coupled, flexible, and testable services.

WHERE SERVICE LAYER DEPENDENCIES COME FROM

Source	Examples
Repositories	Data access operations, CRUD and queries, persistence concerns.
Other Services	Reuse business capabilities, service-to-service calls, orchestration.
Configuration Properties	Application settings, feature flags, environment values.
Security Context	Authentication details, authorization checks, user/role information.
External Systems & Integrations	Third-party APIs, payment gateways, messaging systems.
Utility / Helper Components	Mappers/converters, validators, common utilities.

KEY TAKEAWAY Depend on abstractions, not on concrete implementations. Inject dependencies. Keep services focused and decoupled.

SERVICE DEPENDENCIES (2 OF 2)

Types of dependencies, best practices for managing them, and the impact of doing it well.

TYPES OF DEPENDENCIES

- **Mandatory** — required for the service to function (e.g. Repositories).
- **Optional** — used only in certain scenarios (e.g. NotificationService).
- **Runtime** — resolved and used during execution (e.g. external APIs).
- **Interface-Based** — the service depends on interfaces, allowing easy replacement and testing.
- **External** — dependencies outside the application boundary (e.g. third-party APIs).

DEPENDENCY BEST PRACTICES

- **Depend on Abstractions** — use interfaces for repositories, external clients, and other services.
- **Use Dependency Injection** — inject dependencies via constructor (preferred).
- **Keep Loose Coupling** — avoid tight coupling to concrete classes or static utilities.
- **Minimize Dependencies** — include only what the service needs to fulfill its responsibility.
- **Design for Testability** — easily mock dependencies in unit and integration tests.

Easier to Test

Easier to Change

Improved Maintainability

Better Scalability

Higher Code Quality

KEY TAKEAWAY Well-managed dependencies create clean, maintainable, and enterprise-ready service layers.

WHAT IS A REPOSITORY?

A Repository is a component in the Data Access Layer that abstracts and encapsulates data access logic.

It provides a clean, consistent API for performing CRUD operations and queries on data sources without exposing database details to the Service Layer.

WHAT A REPOSITORY DOES

- **Abstracts the Data Source** — hides the database technology and query implementation.
- **Provides Data Operations** — offers methods to create, read, update, delete, and query data.
- **Promotes Reusability** — centralizes data access logic to avoid duplication.
- **Improves Maintainability** — changes in data access logic are isolated within repositories.
- **Enhances Testability** — repositories can be easily mocked for unit testing.

Analogy: Service <-> Repository <-> DB

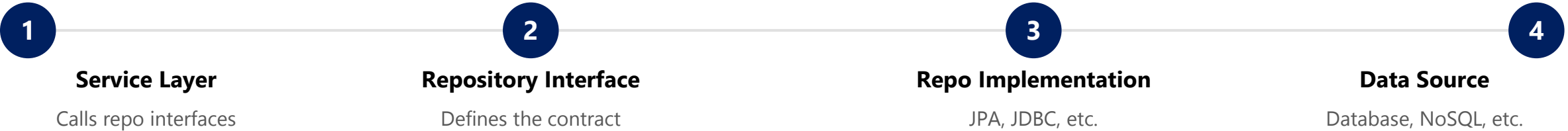
```
// Service asks (like a customer at a store):  
customerRepository.findById("CUS-1001");  
  
// Repository fetches it from storage  
// (like store staff getting it from the back)  
// -- the Service never needs to know how  
//     or where the data is actually kept.
```

KEY TAKEAWAY A Repository provides a collection-like interface for data access, keeping business logic clean and persistence details hidden.

DATA ACCESS LAYER DESIGN (1 OF 2)

The Data Access Layer (Repository Layer) provides a clean abstraction between the application and the database technology.

DATA ACCESS LAYER ARCHITECTURE



DESIGN PRINCIPLES

Principle	Description
Abstraction	Expose interfaces, not implementations. Shield the rest of the app from persistence details.
Single Responsibility	Handle only data access concerns, not business logic.
Loose Coupling	The Service Layer depends on repository interfaces, not concrete classes.
Technology Independence	Design repositories so switching databases is easier.
Consistency & Transactions	Ensure data integrity using proper transaction boundaries.

KEY TAKEAWAY Design the data access layer to isolate persistence details, promote reuse, and enable easy maintenance.

DATA ACCESS LAYER DESIGN (2 OF 2)

Data Access Layer components, best practices, and a Spring Data JPA repository example.

DATA ACCESS LAYER COMPONENTS

Component	Description
Repository Interfaces	Define CRUD and query operations.
Queries	Derived queries, custom queries, or query builders.
Implementations	Concrete data access logic (JPA, JDBC, etc.).
Configuration	Data source, ORM settings, connection pooling.
Entity/Mapping	Map domain objects to database tables.
Exceptions & Translation	Translate persistence exceptions to application exceptions.

AccountRepository.java (Spring Data JPA)

```
public interface AccountRepository
    extends JpaRepository<Account, Long> {

    // Derived query method
    List<Account> findByCustomerId(Long id);

    @Query("SELECT a FROM Account a WHERE a.balance > :amount")
    List<Account> findAccountsWithBalanceGreaterThan(
        @Param("amount") BigDecimal amount);
}
```

BEST PRACTICES

- Use repository interfaces; program to interfaces for flexibility and testability.
- Keep repositories small and focused -- one repository per aggregate/root entity.
- Use meaningful method names; handle transactions at the service layer; avoid returning entities unnecessarily; log and handle data access errors.

KEY TAKEAWAY The service layer uses AccountRepository without knowing how data is accessed or stored -- a well-designed Data Access Layer is clean, maintainable, and easy to evolve.

REPOSITORY RESPONSIBILITIES

A Repository is the bridge between the Service Layer and the data source -- providing a clean, consistent, abstracted way to access data.

WHAT A REPOSITORY IS RESPONSIBLE FOR

- **Data Access Operations** — provide methods to create, read, update, delete, and query data.
- **Query Abstraction** — encapsulate query logic and hide the complexity of the underlying data source.
- **Data Mapping** — map database records to domain entities (and vice versa).
- **Consistency & Integrity** — ensure data integrity using transactions, constraints, and concurrency control.
- **Efficiency** — optimize queries and use pagination, caching, and indexing when needed.
- **Exception Translation** — convert low-level data exceptions into meaningful application exceptions.

WHAT A REPOSITORY SHOULD NOT DO

- Should not contain business logic or validations.
- Should not manage transactions (that's the Service Layer's job).
- Should not know about HTTP requests or responses.
- Should not log business events (use Service/Controller).
- Should not depend on specific database implementation details in the Service Layer.

REMEMBER

Repositories deal with how data is accessed and stored, not what the data means or what business rules apply.

KEY TAKEAWAY A well-defined repository layer leads to clean architecture, higher maintainability, and easier testing.

REPOSITORY BEST PRACTICES (1 OF 2)

Well-designed repositories make your application easier to maintain, test, and evolve.

#	Practice	What It Means
1	Program to an Interface	Define repository interfaces and depend on abstractions, not concrete implementations.
2	Keep Repositories Focused	One repository per aggregate root or entity. Avoid mixing unrelated data concerns.
3	Use Meaningful Method Names	Use names that clearly express intent (e.g. findByEmail, not get).
4	Favor Query Abstraction	Expose queries through method signatures, not raw persistence details.
5	Handle Transactions in the Service Layer	Repositories should not manage transactions -- leave that to the Service.

KEY TAKEAWAY A good repository hides data access complexity, promotes reuse, and keeps your business logic in the Service Layer.

REPOSITORY BEST PRACTICES (2 OF 2)

Practices 6-10, a do's and don'ts table, and a good repository interface example.

PRACTICES 6-10

- **6. Do Not Leak Persistence Details** — never expose EntityManager, Session, or database-specific APIs to upper layers.
- **7. Return Domain Objects or DTOs** — return entities or DTOs, not raw database rows or internal structures.
- **8. Optimize for Performance** — use pagination, indexing, fetch joins, and caching where appropriate.
- **9. Write Testable Repositories** — keep repositories small and use dependency injection to enable easy mocking.
- **10. Log and Handle Data Access Errors** — log meaningful errors and translate low-level exceptions into application exceptions.

DO	DON'T
Use interfaces and DI	Put business logic in repositories
Keep methods small/specific	Return raw SQL results
Use parameter objects for complex queries	Expose persistence framework objects
Use pagination for large result sets	Catch and swallow exceptions

UserRepository.java (Good Example)

```
public interface UserRepository
    extends JpaRepository<User, Long> {

    // Derived query
    Optional<User> findByEmail(String email);

    // Custom query
    @Query("SELECT u FROM User u WHERE u.status = :status")
    List<User> findByStatus(
        @Param("status") UserStatus status);

    // Pagination
    Page<User> findByCreatedDateBetween(
        LocalDateTime start, LocalDateTime end,
        Pageable pageable);
}
```

KEY TAKEAWAY Follow these best practices to build reliable, maintainable, and high-quality data access layers.

CONTROLLER TO SERVICE FLOW

How an incoming request is processed from the Controller through the Service Layer and on to the Repository, and back.



FLOW EXAMPLE (GET /api/customers/123)

- 1. Client sends GET request to /api/customers/123.
- 2. Controller: validates the request, calls `CustomerService.getById(123)`.
- 3. Service: executes business logic (e.g. checks permissions, applies rules).
- 4. Repository: calls `CustomerRepository.findById(123)`.
- 5. Database: returns customer data.
- 6. Service: maps data to a response model.
- 7. Controller: returns a JSON response to the client.

BEST PRACTICE REMINDERS

- Keep Controllers thin -- no business logic.
- Keep Services focused on business rules.
- Use Repositories only for data access.
- Validate input at the Controller level.
- Return consistent and meaningful responses.

KEY TAKEAWAY A clean flow from Controller to Service to Repository ensures scalable, maintainable, and testable applications.

SERVICE TO REPOSITORY FLOW

The Service Layer uses the Repository Layer to perform data access operations, keeping business logic independent of how data is stored.



CustomerService.java (Spring Data JPA style)

```
@Service
public class CustomerService {
    private final CustomerRepository customerRepository;

    public CustomerService(CustomerRepository repo) {
        this.customerRepository = repo;
    }

    public CustomerDTO getCustomerDetails(Long id) {
        Customer c = customerRepository.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("Customer not found"));
        return CustomerDTO.fromEntity(c);
    }
}
```

BEST PRACTICES

- Depend on repository interfaces.
- Expose only necessary methods.
- Use meaningful method names.
- Keep repository calls in services, not controllers.
- Handle exceptions and map to business exceptions.
- Avoid returning entities when DTOs are sufficient.

KEY TAKEAWAY The Service Layer orchestrates business logic and relies on repositories for efficient, secure, and maintainable data access.

REPOSITORY TO DATABASE FLOW

The Repository Layer abstracts the database and handles the details of connecting, executing, and managing data operations.



REPOSITORY RESPONSIBILITIES IN THIS FLOW

- Translate method calls into queries/commands, manage connections through the DataSource.
- Execute database operations and handle transactions (if configured at this level).
- Map results to domain objects or DTOs, and handle exceptions with meaningful translations.
- Return results to the Service Layer.

BEST PRACTICES

Connection Pooling

Parameterized Queries

Fetch Only Required Data

Indexing & Pagination

Log Database Errors

KEY TAKEAWAY The Repository Layer manages all database interactions, returning data to the Service Layer in a consistent and efficient way.

TRANSACTION BOUNDARIES (1 OF 2)

A transaction boundary defines the scope of work that must succeed or fail as a single unit.

Place the boundary in the Service Layer to ensure data consistency and maintain integrity.

Layer	Role in the Transaction	Notes
Controller	Handles HTTP requests only.	Never begins or manages a transaction.
Service (boundary)	Begins the transaction (@Transactional); coordinates repository calls.	Commits if all succeed, rolls back if any fail.
Repository	Executes data access; participates in the transaction.	Never annotated with @Transactional itself.
Database	Commits or rolls back at the end of the transaction.	Enforces durability and consistency.

WHY THIS MATTERS

Data Consistency

Integrity Enforcement

Simplified Error Handling

Maintainability

KEY TAKEAWAY The Service Layer owns the transaction boundary. Repositories participate in the transaction but never control it.

TRANSACTION BOUNDARIES (2 OF 2)

Do's and don'ts for placing transaction boundaries, and a working @Transactional example.

DO'S

- Define transaction boundaries in Service Layer methods.
- Use @Transactional at the Service layer.
- Keep transactions short and focused on a single business use case.
- Let repositories be simple data access components.
- Handle exceptions and let the transaction rollback automatically.

DON'TS

- Do not annotate repository methods with @Transactional (not required).
- Do not start a transaction in the Controller.
- Do not keep transactions open while calling external services.
- Do not perform long-running operations inside a transaction.
- Do not catch and swallow exceptions inside the transaction.

AccountService.java

```
@Service
public class AccountService {
    @Transactional
    public void transfer(Long fromId, Long toId, BigDecimal amount) {
        Account from = accountRepository.findById(fromId).orElseThrow(...);
        Account to = accountRepository.findById(toId).orElseThrow(...);
        from.debit(amount); // may throw exception
        to.credit(amount); // may throw exception
        accountRepository.save(from);
        accountRepository.save(to);
        // If any exception occurs, the transaction rolls back automatically
    }
}
```

KEY TAKEAWAY Define transaction boundaries in the Service Layer to ensure data consistency, integrity, and reliable business operations.

VALIDATION BOUNDARIES (1 OF 2)

Validation ensures data is correct, complete, and within allowed rules before it is processed or persisted.

Validation should occur at the appropriate layer to maintain separation of concerns and provide meaningful feedback.

Layer	What It Validates
Controller	Basic input sanitation (required fields, format). Rejects malformed requests early.
Service (primary boundary)	Business rules and use cases. Ensures data is valid for the domain.
Repository	Enforces data constraints at the persistence level. Ensures integrity and consistency.
Database	Enforces schema constraints and data integrity rules as a final safeguard.

WHY IT MATTERS

Improves Data Quality

Enforces Business Rules

Reduces Errors

Enhances Maintainability

KEY TAKEAWAY Validate early at the Service Layer for business rules, and use Repository constraints for data integrity as a final safeguard.

VALIDATION BOUNDARIES (2 OF 2)

Do's and don'ts for validation placement, and a service-layer validation code example.

DO'S

- Validate at the Service Layer for business rules.
- Use Repository/database constraints for data integrity.
- Return clear, actionable error messages.
- Keep validation logic close to where data is used.
- Use reusable validation rules; log validation failures for monitoring.

DON'TS

- Don't rely only on database constraints.
- Don't duplicate validation logic across many layers inconsistently.
- Don't ignore edge cases and null values.
- Don't expose internal validation details to clients.
- Don't allow invalid data to reach persistence.

CustomerService.java (Service-Layer Validation)

```
public class CustomerService {  
    public CustomerDto createCustomer(CreateCustomerRequest request) {  
        // Business validation  
        if (isBlank(request.getEmail())) throw new ValidationException("Email is required.");  
        if (request.getAge() < 18)  
            throw new ValidationException("Customer must be at least 18 years old.");  
        // ...other business rules  
        return repository.save(customer);  
    }  
}
```

KEY TAKEAWAY Validate early, fail fast, and keep data clean -- ensuring reliability and trust across your application.

ERROR HANDLING BOUNDARIES (1 OF 2)

Error handling boundaries define where exceptions are caught, handled, logged, and translated into meaningful responses.

Place boundaries at the right layers to prevent leakage of internal details and to provide a consistent experience for API consumers.

Layer	Error Handling Role
Controller (API boundary)	Catches exceptions from lower layers; translates to API error responses; returns proper HTTP status codes.
Service (business boundary)	Handles business rule violations, adds context and meaningful messages, throws custom business exceptions.
Repository (data access boundary)	Handles data access errors; wraps low-level exceptions in data exceptions; does not return nulls.
Database (persistence boundary)	Enforces constraints; may raise database errors (e.g. unique key, foreign key violations).

WHY IT MATTERS

- Protects Internal Details
- Consistent Client Experience
- Easier Troubleshooting
- Better Maintainability

KEY TAKEAWAY Catch errors where you can recover or add context. Translate them into clean, consistent error responses at the boundary that communicates with the client.

ERROR HANDLING BOUNDARIES (2 OF 2)

Do's and don'ts for error handling placement, and a Spring global exception handler example.

DO'S

- Define and document an error handling strategy per layer.
- Catch specific exceptions and add context.
- Use custom exceptions for business errors.
- Log errors with correlation IDs.
- Return meaningful, consistent messages to clients; fail fast and avoid swallowing exceptions.

DON'TS

- Don't return stack traces to clients.
- Don't catch and ignore exceptions.
- Don't expose internal or database error codes.
- Don't use generic Exception catch blocks.
- Don't return nulls from repositories; don't mix error handling and business logic.

GlobalExceptionHandler.java (Spring)

```
@RestControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(InvalidAmountException.class)
    public ResponseEntity<ErrorResponse> handleInvalidAmount(InvalidAmountException ex) {
        return ResponseEntity.badRequest().body(new ErrorResponse("VALIDATION_ERROR", ex.getMessage()));
    }
    // ...other @ExceptionHandler methods map each business exception to a status code
}
```

KEY TAKEAWAY Handle errors at the right boundaries, log with context, and return meaningful responses -- no surprises to your clients.

LAB OVERVIEW -- SERVICE AND REPOSITORY LAYERS

Lab 25: Service and Repository Layers with AI Assistance -- Northstar CRM Layering

BUSINESS SCENARIO

- Controllers talking directly to storage create tangled Boot apps that cannot grow transactions, security, or persistence swaps cleanly.
- Leadership freezes the seam: every Customer read/write path is Controller -> Service -> Repository; in-memory storage is fine for now.
- Controllers never import repositories. AI-assisted drafts (Copilot or similar) require a dated, honest review log.

LEARNING OBJECTIVES

- Separate Controller, Service, and Repository responsibilities for Customer CRUD and status updates.
- Define a Spring Data-style CustomerRepository and an in-memory implementation.
- Enforce lifecycle rules (e.g. PROSPECT -> ACTIVE) in the Service, never the Controller.
- Write focused service unit tests with a fake or in-memory repository.
- Explain what changes when the repository later becomes JPA against PostgreSQL.

WHAT YOU BUILD

- Copy prior Boot CRM to lab25-crm; define CustomerRepository and a seeded InMemoryCustomerRepository; put rules in CustomerService; keep CustomerController thin (zero repository imports); wire create/list/status paths; write CustomerServiceTest; log AI review lab25-001; verify dual green mvn test.

KEY TAKEAWAY Fixtures CUS-1001 (Amina Khan, ACTIVE) and CUS-1002 (Ravi Singh, PROSPECT) anchor every example; CUS-9999 proves not-found handling.

LAB TASKS AND DELIVERABLES

Northstar CRM Layering -- implementation steps, deliverables, and the evaluation rubric.

#	Implementation Step
1	Branch prior Boot CRM to lab25-crm; confirm Customer/CustomerStatus compile.
2	Define the Spring Data-style CustomerRepository interface (no JPA yet).
3	Implement seeded InMemoryCustomerRepository (CUS-1001, CUS-1002).
4	Implement CustomerService with duplicate/not-found/transition rules.
5	Build a thin CustomerController -- prove zero repository imports.
6	Wire create, list, and duplicate rejection through all three layers.
7	Unit-test CustomerService with a fake/in-memory repository (no MVC needed).
8	Log AI review lab25-001 + JPA readiness note; run failure experiments + evidence pack.

EXPECTED DELIVERABLES

- Layered Controller -> Service -> Repository source.
- Seeded in-memory repo with CUS-1001 / CUS-1002.
- HTTP evidence + duplicate/not-found failures.
- CustomerServiceTest output; AI review notes (or manual N/A).
- Layering/JPA readiness notes; dual green mvn test; no secrets or target/ committed.

RUBRIC (100 MARKS)

- Environment 10 · Core Impl 30 · Integration/Config 15 · Failure Handling 15 · Verification 10 · Security/AI Review 10 · Docs 10

KEY TAKEAWAY Controllers importing repositories lose core marks; passing tests with only assertNotNull loses verification/AI marks.

MODULE 25 SUMMARY

In this module, we explored how the Service and Repository layers work together to build scalable, maintainable, and testable enterprise applications.

KEY CONCEPTS COVERED



Layered Architecture

Controller, Service, Repository, and Database each own one responsibility.



Service Layer

Business logic, validation, orchestration, and transaction management.



Repository Layer

Clean, abstracted data access that hides persistence details.



Layer Communication

Controller to Service to Repository to Database, and back.



Enterprise Boundaries

Transaction, validation, and error handling boundaries.



Best Practices

Dependency injection, testability, and clean separation of concerns.

MODULE FLOW RECAP



KEY TAKEAWAY By correctly designing and using the Service and Repository layers, we build applications that are robust, testable, and ready for enterprise scale.

KNOWLEDGE CHECK (1 OF 3)

Test your understanding of the Service Layer, transactions, and the Repository Layer's core purpose.

1. What is the primary responsibility of the Service layer?

- A. Handling HTTP requests
- B. Implementing business logic**
- C. Accessing the database tables
- D. Rendering the user interface

3. What is the main purpose of the Repository layer?

- A. Validate input data
- B. Manage business rules
- C. Abstract data access and interact with the database**
- D. Handle exceptions globally

2. Which layer should contain @Transactional in a typical Spring Boot application?

- A. Controller layer
- B. Service layer**
- C. Repository layer
- D. Database layer

4. Which annotation is commonly used in Spring Data JPA repositories?

- A. @Service
- B. @Repository
- C. @Controller
- D. @Component

KEY TAKEAWAY The Service layer owns business logic and transactions; the Repository layer owns data access -- never the reverse.

KNOWLEDGE CHECK (2 OF 3)

Four more questions on validation boundaries, layering violations, request flow, and error handling.

5. At which boundary should input validation ideally occur?

- A. Controller layer
- B. Service layer**
- C. Repository layer
- D. Database layer

7. Which flow correctly represents the processing path of a request?

- A. Controller -> Repository -> Service -> Database
- B. Controller -> Service -> Repository -> Database**
- C. Repository -> Controller -> Service -> Database
- D. Service -> Controller -> Database -> Repository

6. What happens if the Service layer directly uses JDBC or EntityManager instead of a Repository?

- A. It improves performance
- B. It violates separation of concerns**
- C. It reduces code readability
- D. It is required by Spring Boot

8. Where should error handling and exception translation ideally be centralized?

- A. In the Repository layer
- B. In the Service layer
- C. In the Controller layer**
- D. In the Database layer

KEY TAKEAWAY Validate business rules at the Service layer, never bypass the Repository abstraction, and centralize error translation at the Controller boundary.

KNOWLEDGE CHECK (3 OF 3)

Challenge Question: Why is it important to keep the Controller layer thin?

CHALLENGE QUESTION (FROM THE CURRICULUM)

"Why is it important to keep the Controller layer thin?" -- write 3-5 sentences in your own words before revealing the model answer below.

MODEL ANSWER

- **Separation of Concerns** — the Controller's one job is HTTP in, HTTP out; mixing in business logic gives it a second reason to change.
- **Testability** — business rules trapped in a Controller require a full web/MVC test setup instead of a fast, isolated unit test.
- **Reusability** — a thin Controller lets the same business logic be reused by another Controller, a SOAP endpoint, or a batch job -- as Module 24's SOAP endpoint reuses this exact CustomerService.
- **Enforceability** — a thin Controller with zero repository imports is exactly what Lab 25 requires and instructor-verifies by reading the import statements.

KEY TAKEAWAY Understanding the roles, flows, and boundaries of Service and Repository layers helps you build scalable, maintainable, enterprise-ready applications.

"Thin controllers, rule-owning services, and data-hiding repositories -- the backbone of every enterprise Spring app."

MODULE 25 COMPLETE

Service and Repository Layers

You can now design, build, and test layered Spring applications -- with business logic in the Service, data access in the Repository, and clean boundaries for transactions, validation, and errors.